Auto

Language Made for AI OS

```
Language.new("Auto")
```

"从群众中来，到群众中去"

- Made BY AI
- Made FOR AI
- Made OF AI

2026 • 赵普明@索维软件

大纲概览

1. Language as OS
2. Language × AI
3. 融合与突破
4. 行业应用愿景

LaOS

Language as OS

把语言生态当作操作系统来开发，实现去年的愿景。

Language × AI

探索Auto语言与AI的关系

从 **Made by AI** 到 **Made For AI**

AI + OS

融合与突破

AI与OS的深度融合，打破不可能三角限制，实实现开发效率、程序安全与运行性能的全面突破。

Write Once, Run Anywhere

行业应用愿景

从单一OS到跨行业应用，Auto语言将扩展至汽汽车、机器人和通用应用开发

```
auto::outline("2026")
```

Intro – Auto: 编程语言开放社区（PLOC）



AUTO LANG

编程语言开放社区

Intro – Auto: Simpler Rust , Better C

Simpler Rust
提高开发效率

- **Script mode with AutoVM interpreter**
  脚本模式，无需编译
- **Simpler Lifetime**
  简化生命周期管理
- **Simpler Generics**
  更简洁的泛型语法
- **Simpler Meta programming**
  类Zig的元编程体验

Better C
增强语言能力

- **C + OOP**
  C语言 + 面向对象特性
- **Module system**
  现代模块系统
- **Generics and CTE**
  泛型 + 编译期执行
- **Safety**
  内存安全保障

Intro – 编程语言的不可能三角

⚠ 传统不可能三角

开发效率、程序安全、运行性能三者不可兼得，传统语言必须在其中两个方面做出妥协。

Auto 的突破

开发效率

- AutoVM 解释器
- AIE 增量编译

程序安全

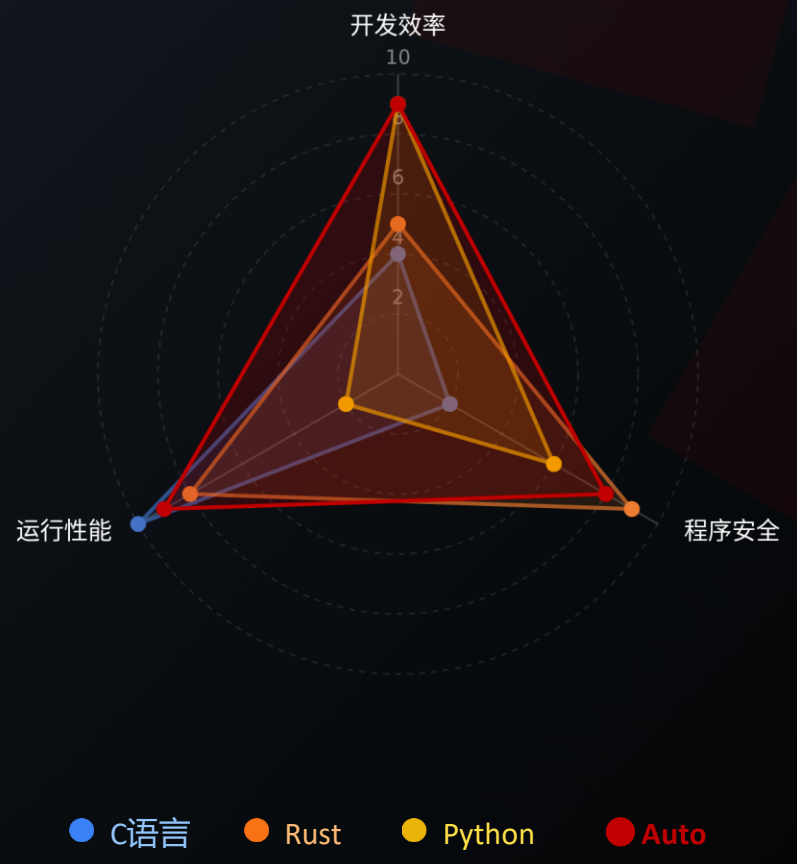
- a2rs Rust语言基底
- Lifetime 生命周期

生态壁垒

- a2c C语言转译器
- a2py Python语言转译器
- a2ts TypeScript语言转译器
- a2kt Kotlin语言转译器

四语言对比 (10分制)

语言	开发效率	程序安全	运行性能
C	4	2	10
Rust	5	9	9
Python	9	6	2
Auto	**9**	**8**	**9**



auto::突破不可能三角

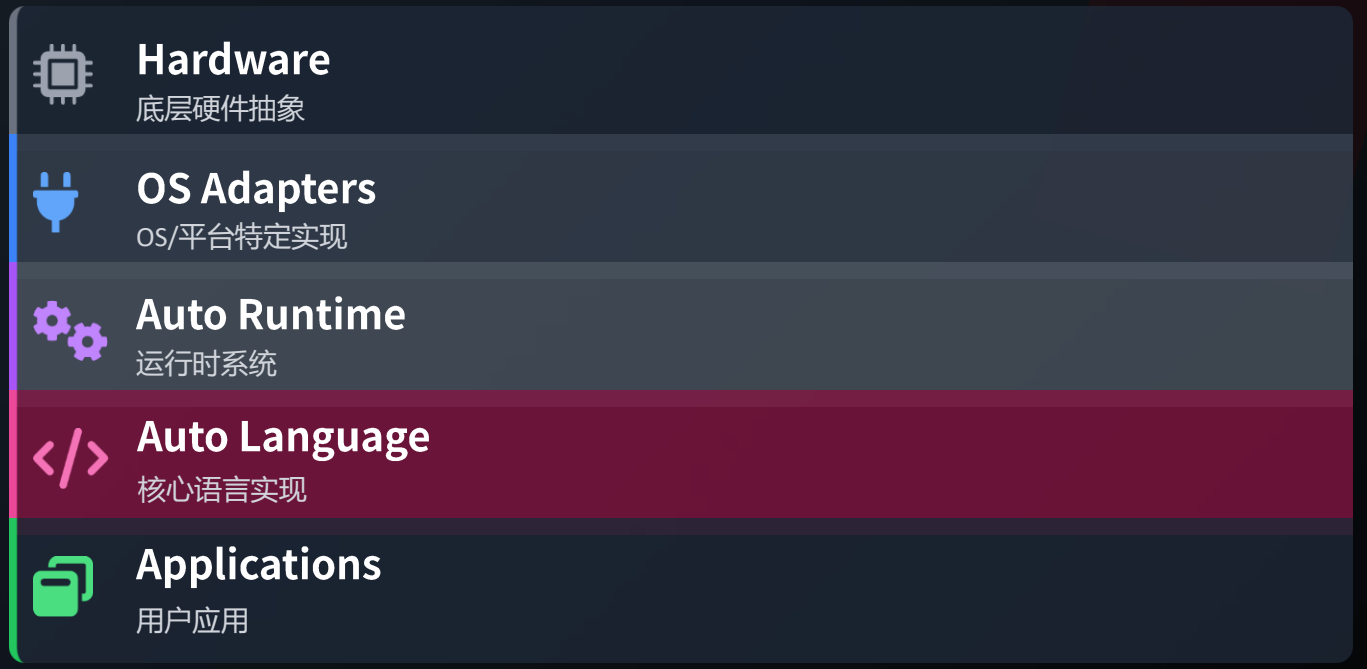
Intro - LaOS (Language as OS)

语言即操作系统

LaOS (Language as OS) 是一种新型计算架构，
将编程语言提升为操作系统层面的抽象。

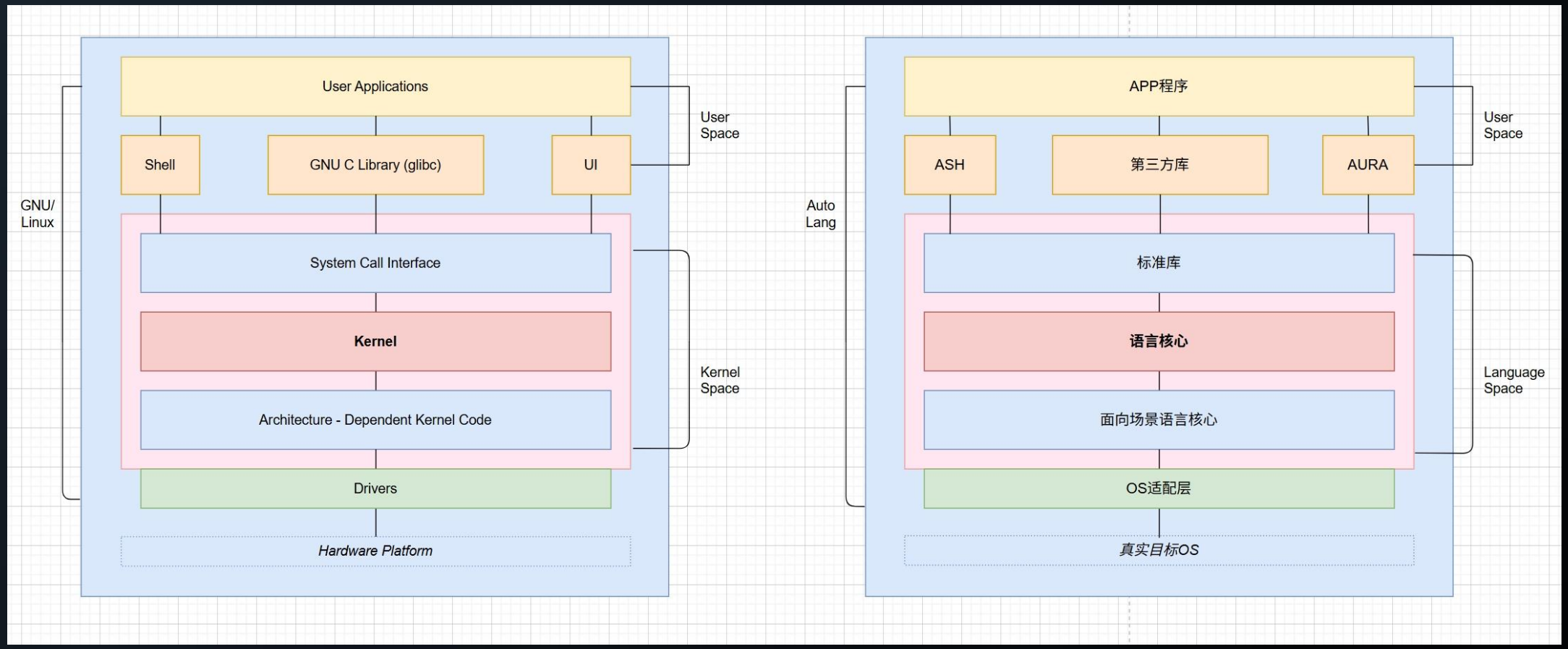
核心突破

- 突破传统"语言+运行时+OS"架构
- Auto作为应用/OS/硬件的桥梁
- 统一系统编程模型



Intro - LaOS (Language as OS)

语言即操作系统



Linux

Auto Lang

Progress - LaOS 内核（语言特性）

AutoVM

standalone

独立运行模式，无需依赖外部解释器

comptime

编译期执行，提前计算表达式结果

embedded

前端在上位机，VM核心在MCU中

转译器矩阵

a2rs	→ Rust
a2c	→ C
a2ts	→ TypeScript
a2py	→ Python
a2kt	→ Kotlin

高阶特性

CTE

编译期执行 (Compile-Time Execution)

Spec

特征系统 (Trait System)

OOP

`is | has | as`

```
auto::features("Auto")
```

Progress – CTE：编译期调用AutoVM解释执行

编译器代码块调用

```auto
// 编译期计算斐波那契数列的第 20 项
fn fib(n int) int {
    if n <= 1 {
        return n
    }
    return fib(n - 1) + fib(n - 2)
}

// 编译期执行代码块，结果嵌入到运行时代码中
const FIB_20 int = #{ fib(20) }

```

除#外，CTE代码和普通Auto代码一致

编译期条件执行

```auto
// 根据编译期条件生成不同代码
#is get_target_os() {
    "windows" => {
        fn get_path_sep() str {
            return "\\"
        }
    }
    "linux" => {
        fn get_path_sep() str {
            return "/"
        }
    }
    else => {
        fn get_path_sep() str {
            return "/"
        }
    }
}
```

Progress – OOP：spec，as，has，is，for

特征（Spec）

```
spec Calculator {
    fn add(a int, b int) int
    fn multiply(a int, b int) int
}

type MathEngine as Calculator {
    fn add(a int, b int) int {
        a + b
    }

    fn multiply(a int, b int) int {
        a * b
    }
}
```

组合（Composition）

```
type Computer {
    has engine MathEngine for Calculator
}

fn main() {
    let comp = Computer()
    // Now use comp as a Calculator
    let result1 = comp.add(5, 3)
    let result2 = comp.multiply(4, 7)
    print(result1)
    print(result2)
}
```

Progress - 内存管理

核心问题：动态内存 vs 静态内存?

动态内存

- ✓ 灵活性高，运行时分配
- ✗ 内存安全风险
- ✗ 手动管理繁琐

Auto 的统一方案

分配：动静结合

回收：AutoFree

静态内存

- ✓ 性能高，编译时确定
- ✗ 灵活性受限
- ✗ 难处理动态数据结构

Default Storage（默认存储策略）

根据变量特性自动选择最佳内存分配策略，无需开发者干预。

```
10  type List<T, S = DefaultStorage> {
11      // Number of elements currently in the list
12      len u32
13
14      // Storage backend (Heap<T>, Inline<T, N>, etc.)
15      // Must implement Storage<T> spec
16      store S
17
```

```
auto::memory("management")
```

Prelude Config（预置配置）

对于不同系统，提供预制的内存管理配置

```
34  const DefaultStorage = HeapStorage
35
36  use auto.list: List
37  use auto.hashmap: HashMap
38  use auto.hashset: HashSet
39  use auto.vecdeque: VecDeque
40  use auto.btreemap: BTreeMap
```

```
96  fn main() {
97      // 开发者无需提供Storage策略，使用系统默认策略
98      let xs = List.new([1, 2, 3, 4])
99  }
```

Progress - 异步并发

核心抽象：Actor模型

原生支持异步编程模型

通过~T类型表示异步计算，编译器自动处理复杂的状态机转换与协程生成。

Task/Msg

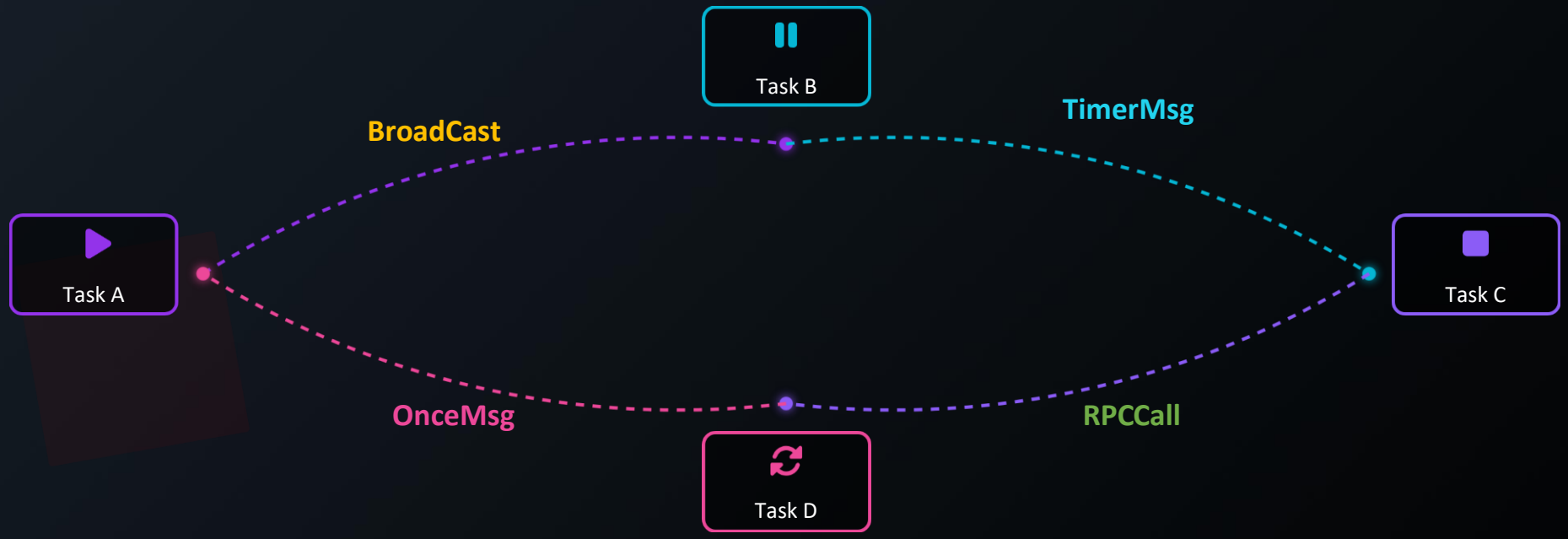
消息传递机制

Task间通过消息传递共享数据，避免共享内存带来的复杂同步问题与数据竞争。

异步Future：~T

无数据竞争的并发

通过所有权系统和消息传递，在编译期消除数据竞争，保证程序安全性。



Progress - 异步并发

单个Future

```
// 假设这个函数的返回是~User，它就是一个异步函数
fn fetch_user(id int) ~User {
    // ...
}

// 单个Future，等待结果，此时user1的类型是User
let user1 = fetch_user(1).await

// 单个Future，不等待结果，此时`.go`返回的仍然是个Future
let future2 = fetch_user(2).go

// 此时可以去做其他的事情
let user3 = fetch_user(3).await

// 最后，等待future2结束
let user2 = future2.await
```

多个Future

```
// 抢最快
let fast = [cdn_a(), cdn_b()].first_resp().?

// 要一个成功的
let data = [server_a(), server_b()].first_ok().?

// 等待所有的结果都返回，100ms超时，可能有失败结果
let users = [
    create_user("alice"),
    create_user("bob"),
    create_user("charlie"),
].all_resp(timeout: 100)

// 等待所有的结果都成功，最大重试次数为3，结果都为成功
let users = [
    create_user("alice"),
    create_user("bob"),
    create_user("charlie"),
].all_ok(retry: 3)
```

Progress - 标准库

io.at
统一接口

io.vm.at
虚拟机后端

io.rs.at
Rust 后端

io.c.at
C 后端



- **一次编写，多平台运行**
  统一API，可编译至不同平台，实现"Write Once, Run Anywhere"
- **平台特定优化**
  针对不同平台特性进行优化，提供最佳性能
- **灵活后端选择**
  根据目标平台自动选择最合适的后端实现

```
io::backend("stdlib")
```

Progress - Shell（ASH）与 UI（Aura）

Auto Shell (ASH)

```
ash: AutoShell
```

基于 nushell + uutils/coreutils
继承强大的Posix跨平台命令实现

结构化管道 (Pipeline with Atom)
采用Atom/Batom格式（JSON的优化版）作为管道的数据格式

跨边界通信 (MCU <-> Windows)
实现不同系统间的无缝数据交换与命令执行

智能命令 (Smart Commands)
AI辅助的命令补全与自动建议功能

未来扩展: SafeClaw
计划扩展至SafeClaw，提供更安全的系统级操作

Auto UI (Aura)

```
aura: Auto UI Representation Abstract
```

多平台目标

Web
Vue3 + shadcn-vue

Desktop
Tauri Desktop
Rust (iced/gpui)

Mobile
Jetpack Compose / Harmony
Harmony
HarmonyOS/ArkTS

灵活后端设计

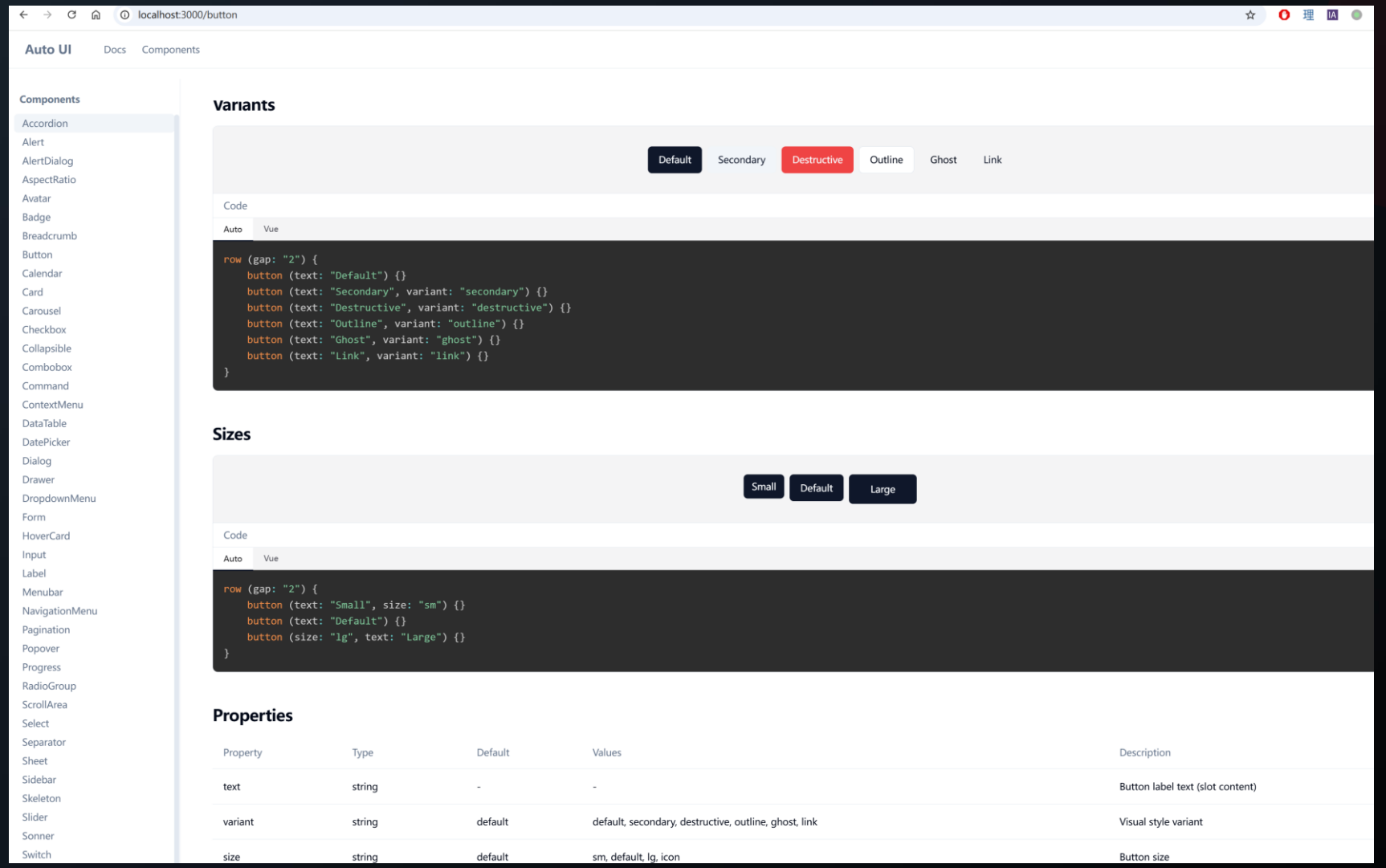
Web → Rust HTTP Server 轻松实现前后端分离

Desktop → Rust IPC 高效的桌面应用交互

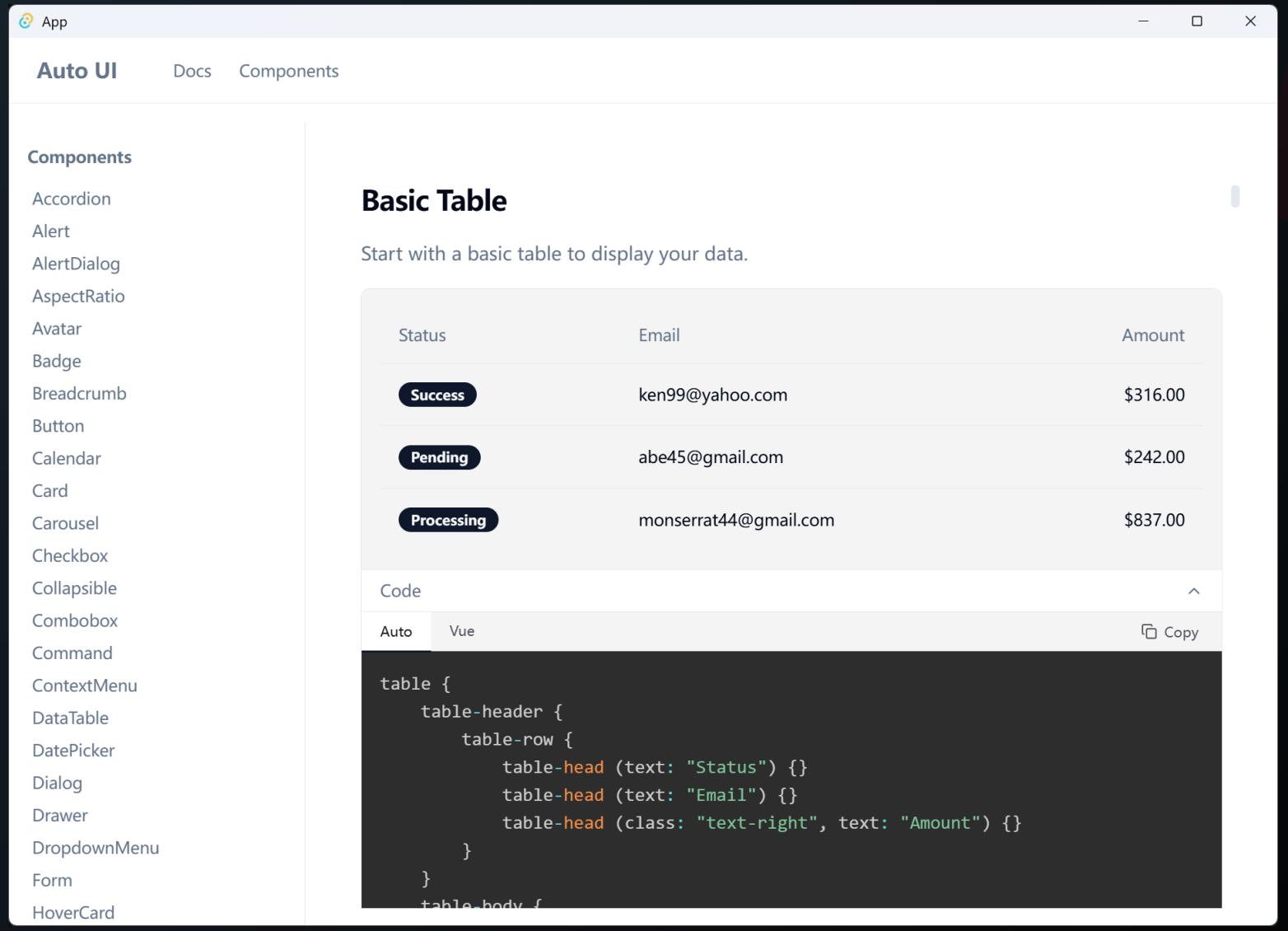
MCU → C + LVGL (未来) 支持嵌入式设备界面

```
auto::shell("ash") auto::ui("aura")
```

Progress – AURA Demo (Vue.js)



Progress – AURA Demo (Tauri Desktop)

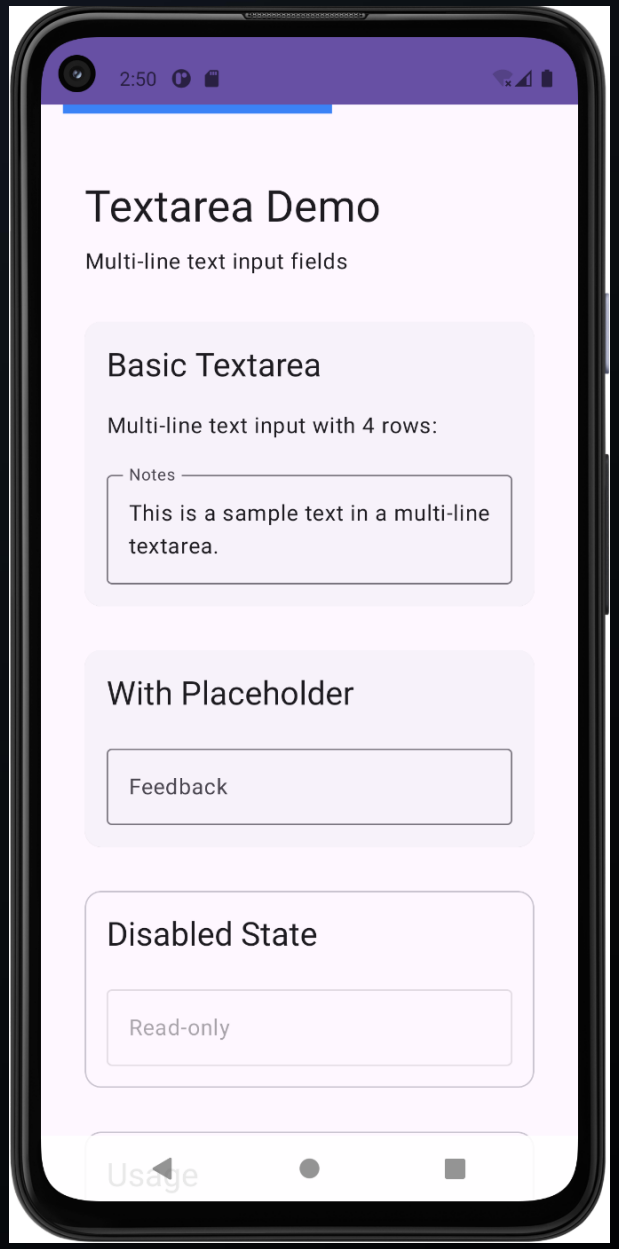


Progress – AURA Demo (JetPack Compose)

```
Scroll {
    Col {
        style: "p-8 gap-8"
        // Header
        Col {
            style: "gap-2"
            H1 "Textarea Demo"
            Text (variant: "muted") "Multi-line text input fields"
        }

        // Basic Textarea
        Card (variant: "elevated") {
            style: "p-4 rounded-xl w-full"
            Col {
                style: "gap-4"
                H3 "Basic Textarea"
                Text "Multi-line text input with 4 rows:"
                textarea (value: .notes, label: "Notes", rows: 4)
            }
        }

        // With Placeholder
        Card (variant: "elevated") {
            style: "p-4 rounded-xl w-full"
            Col {
                style: "gap-4"
                H3 "With Placeholder"
                textarea (placeholder: "Enter your feedback here...", label: "Feedback", rows: 3)
            }
        }
```



Progress – AURA Demo (Harmony/ArkTS)

```
/// Quick start page with Banner, Grid, and List
widget QuickStartPage {
    view {
        Col {
            Text "快速入门" {
                style: "text-2xl font-bold w-full text-left pl-4"
            }
            Scroll {
                Col {
                    Banner {}
                    EnablementView {}
                    TutorialView {}
                }
                style: "flex-1"
            }
            style: "w-full h-full bg-gray-100"
        }
    }

    routes {
        "articleDetail" => use ArticleDetailPage
        "bannerDetail" => use BannerDetailPage
    }
}

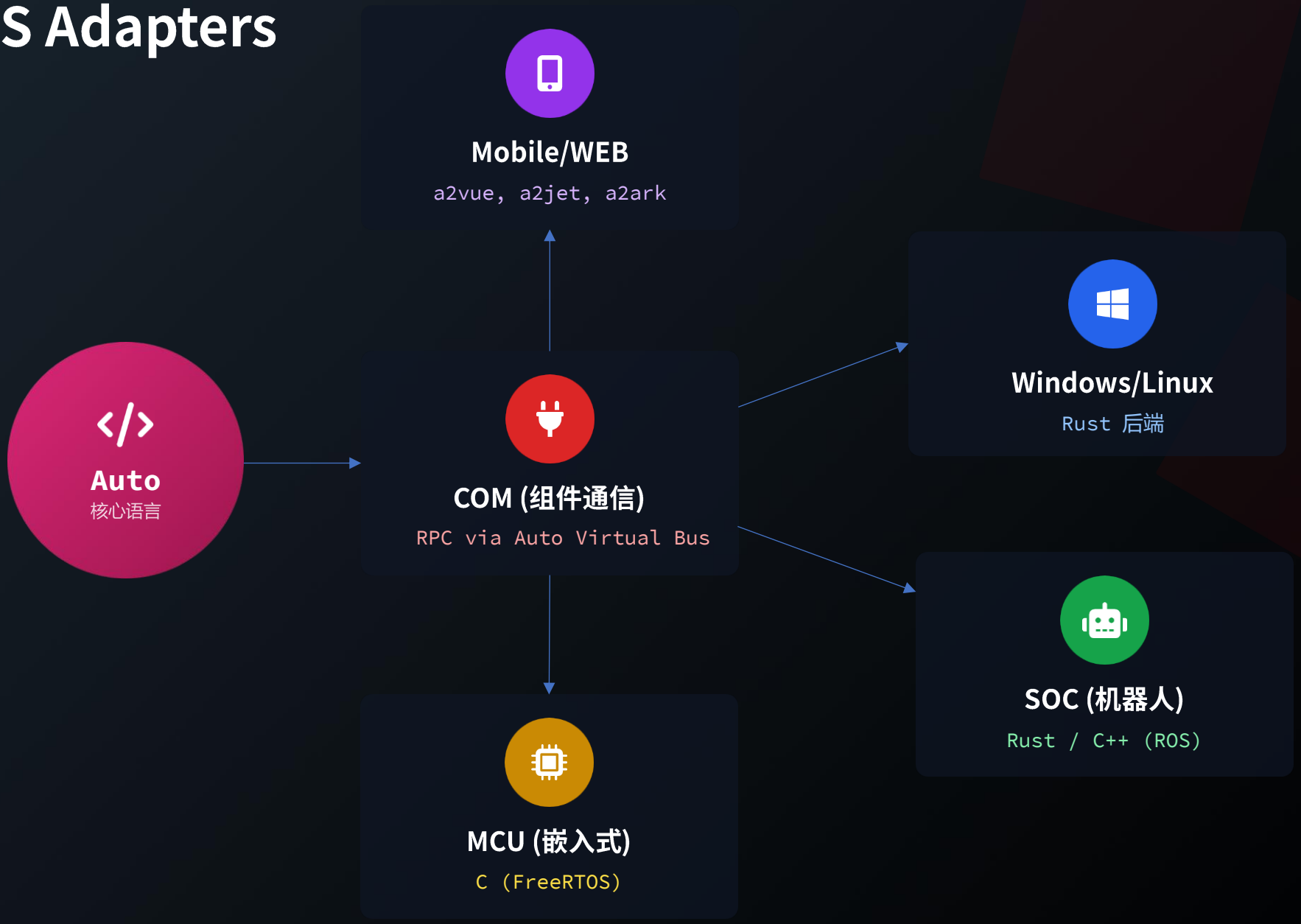
```



Progress – AURA Demo (Rust/GPUI)

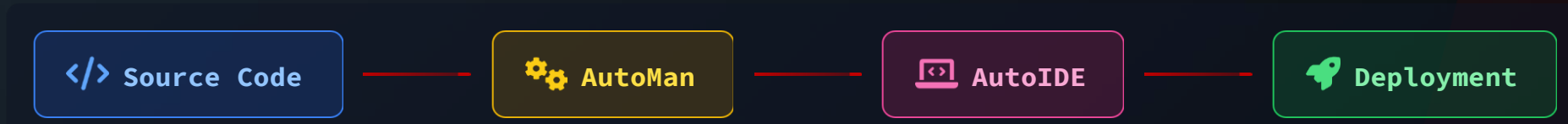


Progress - OS Adapters




```
auto::adapters("多平台适配")
```

Progress - DevOp & Distribution



AutoMan 通用项目管理器

构建系统

自动化编译流程，支持多平台交叉编译，智能依赖解析与增量构建。

包管理

统一依赖管理，支持SemVer版本控制，自动解决依赖冲突，提供私有与公共仓仓库。

```
automan build --target vue
```

AutoIDE 集成开发环境

Assemble them all

统一集成各类工具链，自动配置开发环境，支持多语言混合开发，提供统一调试界面。

智能辅助

AI驱动的代码补全，智能重构，实时错误分析，自动格式化与优化。

```
autoide . --no-sandbox
```

```
auto::devop(“auto-ide")
```

大纲概览

1. Language as OS
2. Language × AI
3. 融合与突破
4. 行业应用愿景

LaOS

Language as OS

把语言生态当作操作系统来开发，实现去年的愿景。

Language × AI

探索Auto语言与AI的关系

从 **Made by AI** 到 **Made For AI**

AI + OS

融合与突破

AI与OS的深度融合，打破不可能三角限制，实实现开发效率、程序安全与运行性能的全面突破。

Write Once, Run Anywhere

行业应用愿景

从单一OS到跨行业应用，Auto语言将扩展至汽汽车、机器人和通用应用开发

```
auto::outline("2026")
```

Outline – Auto: AI 三位一体



Made BY AI

由AI主导开发

基于100亿Tokens的AI算力，两个月内完成成传统需要数年的开发工作，由AI主导语言设计与实现。

Made FOR AI

为AI代码生成优化

专为AI生成设计的语法，双模式错误信息，注解式契约编程，让AI更容易理解和生成代码。

Made OF AI

由AI代码构成

语言本身由AI代码构成，AI可以直接理解和修改Auto代码，实现"一次生成，处处使用""的愿景。

```
auto::ai("triad")
```

Progress - Made BY AI （两个月开发奇迹）

开发周期

两个月完成传统需数年的工作

2 个月

AI算力

智谱AI MAX套餐

100亿 tokens

代码产出

高质量Rust代码实现

1300+ commits

20万 行Rust

交付成果

1个解释器

完整实现Auto语言核心功能

6个转译器

a2c, a2rs, a2ts, a2py, a2jet, a2ark

语言特性实现

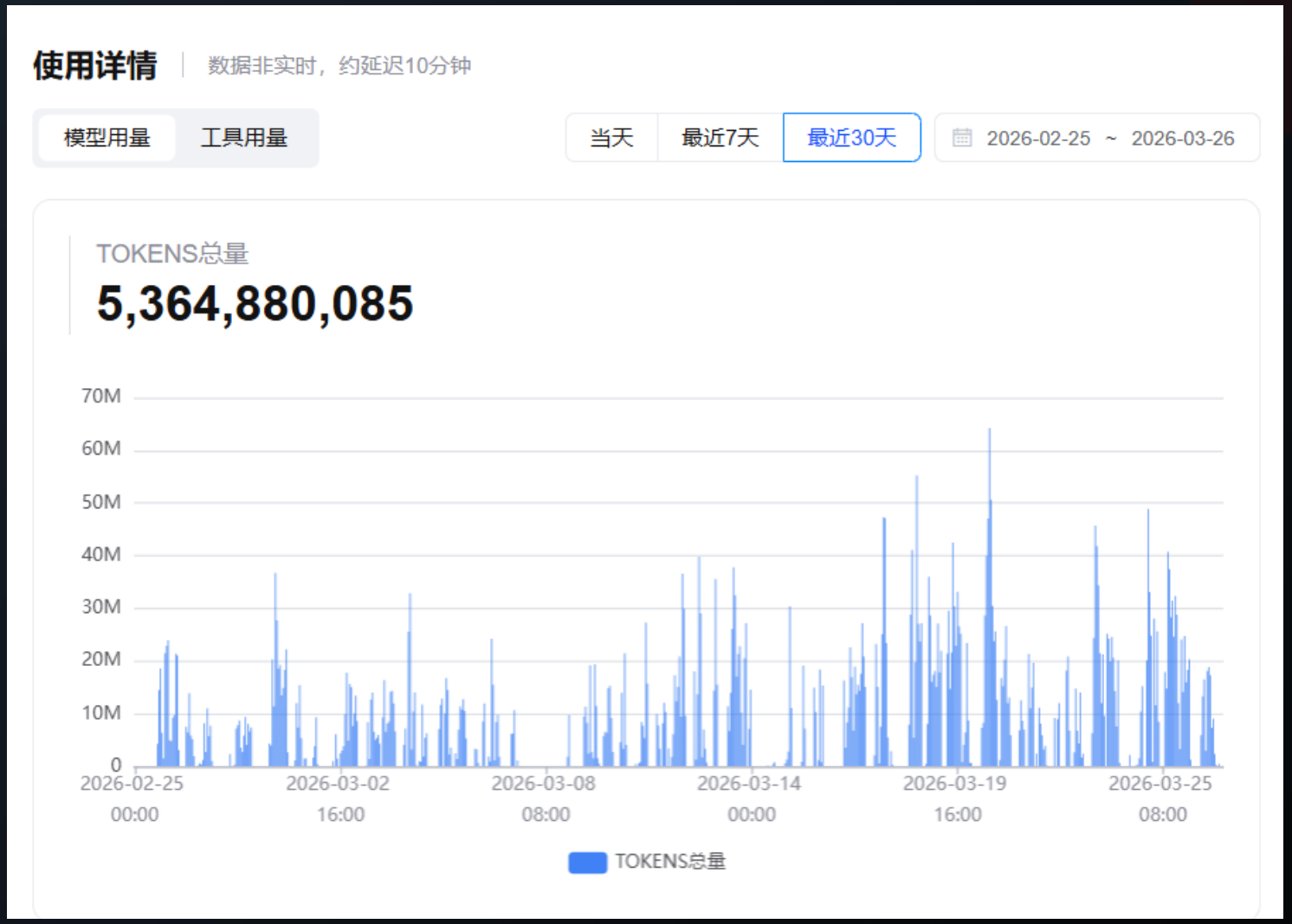
a2c (C语言支持)	90%
a2rs (Rust语言支持)	80%
Auto语法实现	95%

4个生态工具

UI（AURA），Shell（ASH），Builder（AutoMan），Doc（AutoDown/AutoGen）（AutoDown/AutoGen）

```
auto::build(" AI miracle ")
```

Progress - Made BY AI (GLM5 Token用量)



Outline – Auto: AI 三位一体



Made BY AI

由AI主导开发

基于100亿Tokens的AI算力，两个月内完成传统需要数年的开发工作，由AI主导语言设计与实现。

Made FOR AI

为AI代码生成优化

专为AI生成设计的语法，双模式错误信息，注解式契约编程，让AI更容易理解和生成代码。

Made OF AI

由AI代码构成

语言本身由AI代码构成，AI可以直接理解和修改Auto代码，实现"一次生成，处处使用""的愿景。

```
auto::ai("triad")
```

AI - Made FOR AI



AI友好的语法设计

AI × 人类 = Auto

双模式错误信息（--ai flag）

人类:
miette（美观的错误展示）

AI:
JSON（结构化错误信息）

注解式契约编程

```
#[in(…)]
#[out(…)]
#[invariant(…)]
fn divide(a: i32, b: i32) -> i32
```

意图注解

```
#[intent("生成REST API客户端")]
 fn create_client()
```

AI示例代码库

- 区分对人和对AI的Demo示例
- AI辅助生成大规模Demo库

反向转译

- 将目标代码工程反向生成Auto代码
- 理解人类注释，生成符合AI理解的注释

Claude Skill

- Auto代码生成技能
- 集成开发环境

```
auto::ai("friendly")
```

Outline – Auto: AI 三位一体



Made BY AI

由AI主导开发

基于100亿Tokens的AI算力，两个月内完成成传统需要数年的开发工作，由AI主导语言设计与实现。

Made FOR AI

为AI代码生成优化

专为AI生成设计的语法，双模式错误信息，注解式契约编程，让AI更容易理解和生成代码。

Made OF AI

由AI代码构成

语言本身由AI代码构成，AI可以直接理解和修改Auto代码，实现"一次生成，处处使用""的愿景。

```
auto::ai("triad")
```

AI - Made OF AI： 自制 Agent 架构



AI - Made OF AI： AI 编译加速

Interact with AI Languages

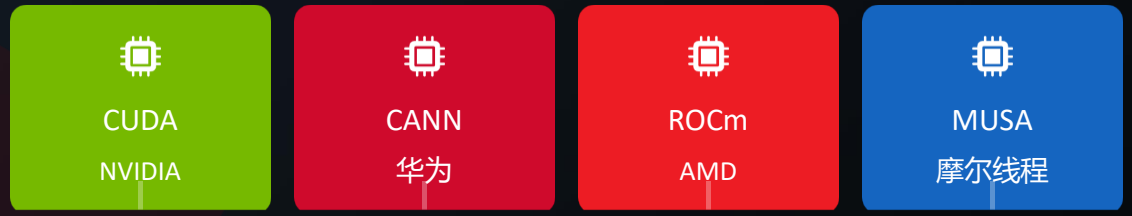


GPU通用计算



高性能AI推理

直接转译层 - Direct transpilation



Auto直接转译到各种计算平台

大纲概览

1. Language as OS
2. Language × AI
3. 融合与突破
4. 行业应用愿景

LaOS

Language as OS

把语言生态当作操作系统来开发，实现去年的愿景。

Language × AI

探索Auto语言与AI的关系

从 **Made by AI** 到 **Made For AI**

AI + OS

融合与突破

AI与OS的深度融合，打破不可能三角限制，实实现开发效率、程序安全与运行性能的全面突破。

Write Once, Run Anywhere

行业应用愿景

从单一OS到跨行业应用，Auto语言将扩展至汽汽车、机器人和通用应用开发

```
auto::outline("2026")
```

AI + OS - Agent OS 生态

Cursor

初始阶段：AI驱动的代码编辑器，提供基础AI辅助编程体验。

Claude

进化阶段：通过Agent和Skill，提供供更智能的开发环境。

OpenClaw



突破阶段：多Agent体系，通过tools/skills实现跨平台的AI工作环境境

Agent OS

整合目标：由Agent组成的操作系统，，更安全高效的分布式智能体协作网络。

多Agent框架的探索

- OpenFang：更严谨的多Agent体系，初步探索Agent OS
- 三省六部制：新颖的Agent治理架构
- IronClaw：安全优先的OpenClaw，使用Rust实现

SafeClaw

- 从NanoClaw转译成Rust，再反转译成Auto
- 未来会融合IronClaw特性，增强安全性
- 集成 Auto UI，并嵌入到Auto语言的虚拟OS系统内

```
agent_os::evolution("Cursor → Claude → OpenClaw → Agent OS")
```

AutoOS - 跨平台虚拟操作系统



跨平台虚拟OS

Win / Linux / Mac / Android / Harmony / MCU

Agentic Apps

统一应用服务，统一通讯架构

全流程支持

AutoIDE → AutoUI → AutoMan（开发到部署）

开放生态

OpenSource / OpenBusiness

大纲概览

1. Language as OS
2. Language × AI
3. 融合与突破
4. 行业应用愿景

LaOS

Language as OS

把语言生态当作操作系统来开发，实现去年的愿景。

Language × AI

探索Auto语言与AI的关系

从 **Made by AI** 到 **Made For AI**

AI + OS

融合与突破

AI与OS的深度融合，打破不可能三角限制，实实现开发效率、程序安全与运行性能的全面突破。

Write Once, Run Anywhere

行业应用愿景

从单一OS到跨行业应用，Auto语言将扩展至汽汽车、机器人和通用应用开发

```
auto::outline("2026")
```

Beyond Single OS - 跨行业应用展望

汽车行业

整车开发

- 车载系统一体化开发
- 座舱APP: a2jet/a2ark
- SOC软件：a2rs/a2c++
- MCU底软：a2c
- 车内通讯：vbus for (ETH/CAN/LIN/..)

机器人行业

AutoOS = ROS++

- 主控芯片：a2rs/a2c++
- 执行器：a2c
- 大小脑：a2py

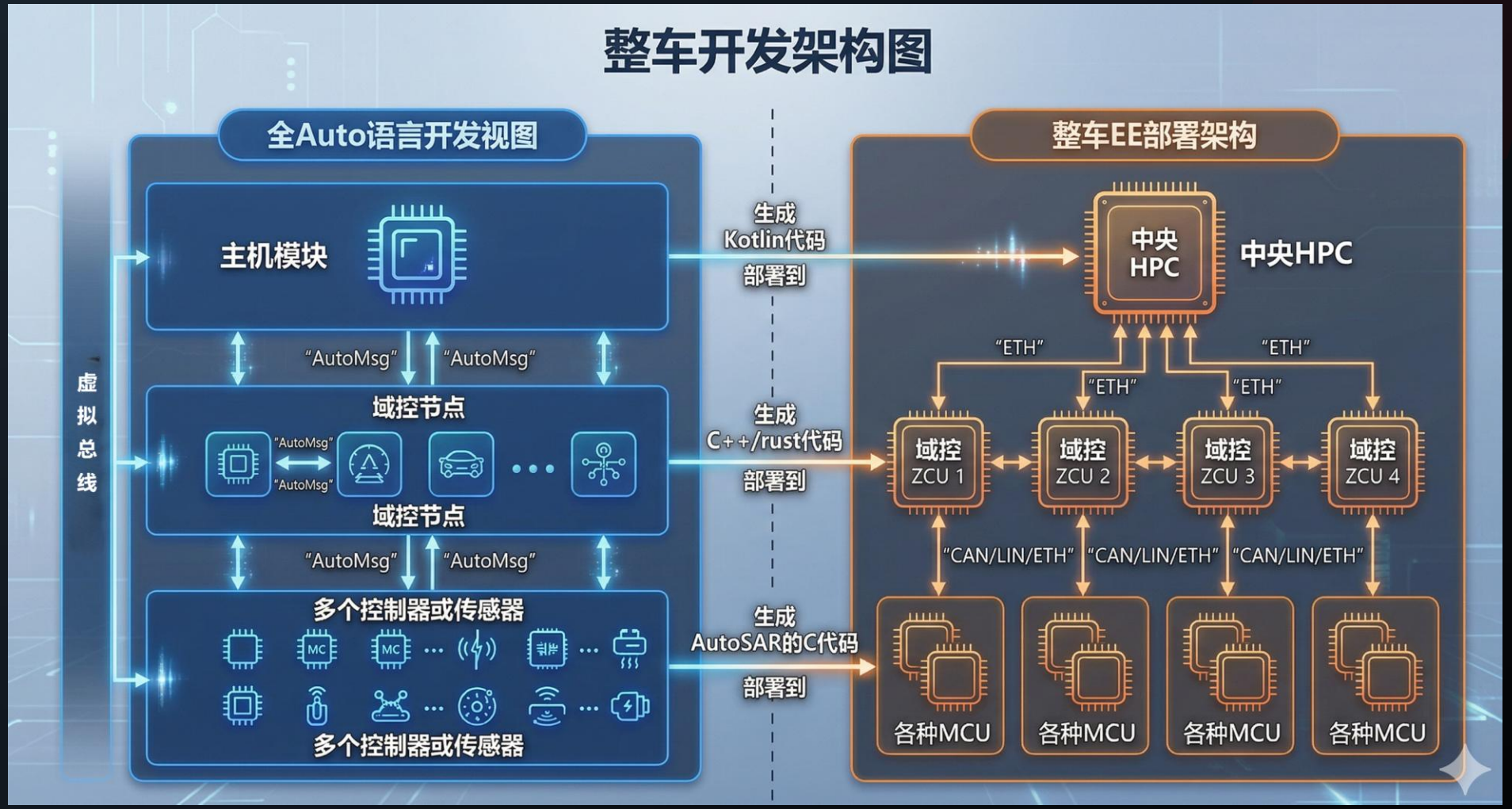
应用开发行业

真全栈开发

- 前后端统一开发
- 跨平台UI框架
- 后端服务与数据库集成
- 自适配跨端调用

```
auto::cross_platform()
```

Beyond Single OS - 跨行业应用展望



Q&A

项目仓库

gitee.com/auto-stack/auto-lang
gitcode.com/auto-stack/auto-lang
github.com/auto-stack/auto-lang

微信群

Auto语言讨论群

公众号

PLOC国产语言联盟

感谢您的关注

我们期待与您深入交流，共同探讨Auto语言的未来

Made BY AI , FOR AI , OF AI